

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

C02: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze
8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema	BL5 – Evaluate

9	and suggest appropriate integrity constraints.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Normalize a University Student–Course Table up to 3NF Scenario

A university stores all student and course data in a single flat table. This is analyzed for normalization issues and redesigned up to 3NF.

Assumed Unnormalized Table

StudentCourse (StudentID, StudentName, StudentDept, CourseID, CourseName, Instructor, InstructorOffice, Grade)

Primary key: (StudentID, CourseID) – one row per student per enrolled course.

Functional Dependencies

StudentID → StudentName, StudentDept
CourseID → CourseName, Instructor
Instructor → InstructorOffice
(StudentID, CourseID) → Grade

Step 1 – 1NF

All attributes hold single atomic values and each row is uniquely identified by (StudentID, CourseID), so the table already satisfies 1NF.

Step 2 – 2NF (remove partial dependencies)

The primary key is composite: (StudentID, CourseID). StudentName and StudentDept depend only on StudentID (part of the key), and CourseName and Instructor depend only on CourseID (part of the key). These are partial dependencies, so the table violates 2NF.

Step 3 – 3NF (remove transitive dependencies)

Even after removing partial dependencies, InstructorOffice depends on Instructor, and Instructor depends on CourseID – a transitive dependency (CourseID → Instructor → InstructorOffice) that must also be removed.

Final 3NF Decomposition

Relation	Attributes	Key
Student	StudentID, StudentName, StudentDept	PK: StudentID
Course	CourseID, CourseName, InstructorID (FK)	PK: CourseID
Instructor	InstructorID, InstructorName, InstructorOffice	PK: InstructorID
Enrollment	StudentID (FK), CourseID (FK), Grade	PK: (StudentID, CourseID)

Justification

Every non-key attribute in each relation now depends on the whole key of that relation, and only that key — no partial or transitive dependencies remain, so all four relations are in 3NF.

Redundancy is eliminated: student details are stored once regardless of how many courses they take, and course/instructor details are stored once regardless of how many students enrol.

Update/insertion/deletion anomalies are removed — e.g., a course's instructor can be changed in one place, and a new course can be added without needing an enrolled student.

Q2. SQL to Retrieve Employees Working in Multiple Departments

Scenario

A company database holds employee and department details. Employees can work in more than one department, so a query must retrieve employees associated with multiple departments, with a justified schema design.

Assumed Schema

A plain Employee(EmpID, EmpName, DeptID) table can only model one department per employee (a 1:N relationship). Since an employee may belong to several departments, an associative (junction) table is required to model the underlying many-to-many relationship:

```
Employee    (EmpID PK, EmpName)
Department  (DeptID PK, DeptName)
EmployeeDept (EmpID FK, DeptID FK, RoleInDept)
             PK: (EmpID, DeptID)
```

SQL Query

```
SELECT e.EmpID, e.EmpName, COUNT(DISTINCT ed.DeptID) AS DeptCount
FROM   Employee e
JOIN   EmployeeDept ed ON e.EmpID = ed.EmpID
GROUP BY e.EmpID, e.EmpName
HAVING COUNT(DISTINCT ed.DeptID) > 1;
```

To also list which departments each such employee belongs to:

```
SELECT e.EmpID, e.EmpName, d.DeptName
FROM   Employee e
JOIN   EmployeeDept ed ON e.EmpID = ed.EmpID
JOIN   Department d ON ed.DeptID = d.DeptID
WHERE  e.EmpID IN (
```

```

SELECT EmpID FROM EmployeeDept
GROUP BY EmpID HAVING COUNT(DISTINCT DeptID) > 1
)
ORDER BY e.EmpID;

```

Justification

A junction table (EmployeeDept) is necessary because a foreign key placed directly on Employee can only reference one DeptID at a time; it cannot represent an M:N relationship without violating 1NF (a repeating DeptID list) or 2NF.

GROUP BY with HAVING COUNT(DISTINCT DeptID) > 1 is the set-based way to isolate employees appearing against more than one distinct department – more efficient than a self-join or a procedural loop.

The second query reuses the same aggregate as a filter (IN subquery) so department names can be listed per row without collapsing them under GROUP BY.

Q3. Candidate Keys and BCNF Check/Decomposition

Scenario

Given a relation with multiple functional dependencies, determine all candidate keys, check whether the relation satisfies BCNF, and decompose it if it does not.

Given Relation

R(A, B, C, D) with functional dependency set F:

F = { AB → C, C → D, D → A }

Step 1 – Attribute Closures (all 15 non-empty subsets tested)

Subset	Closure	Superkey?
A	{A}	No
B	{B}	No
C	{A,C,D}	No
D	{A,D}	No
AB	{A,B,C,D}	Yes
AC	{A,C,D}	No
AD	{A,D}	No
BC	{A,B,C,D}	Yes

BD	{A,B,C,D}	Yes
----	-----------	-----

Step 2 – Candidate Keys

The minimal attribute sets whose closure is {A,B,C,D} are AB, BC and BD. Each is minimal (no single attribute alone reaches the full closure), so:

Candidate Keys = { AB, BC, BD }

Every attribute (A, B, C, D) belongs to at least one candidate key, so all four attributes are prime – this relation is already in 3NF.

Step 3 – BCNF Check

FD	Closure of LHS	LHS a superkey ?	BCNF holds for this FD?
AB → C	{A,B,C,D}	Yes	Yes
C → D	{A,C,D}	No	No – violates BCNF
D → A	{A,D}	No	No – violates BCNF

R is NOT in BCNF (C → D and D → A both have a non-superkey determinant), even though it is in 3NF. This is decomposed below.

Step 4 – BCNF Decomposition (on violation C → D)

Split into R1(C, D) and R2(A, B, C):

R1(C, D) – FD: C → D. C is now the key of R1 → in BCNF.

R2(A, B, C) – projected FDs: AB → C, and C → A (derived by transitivity, since C → D → A in the original set).

Candidate keys of R2: AB and BC.

Check: AB → C, AB is a superkey → OK

C → A, C is NOT a superkey → still violates BCNF

Step 5 – Further Decomposition of R2 (on violation C → A)

Split R2(A, B, C) into R3(A, C) and R4(B, C):

R3(A, C) – FD: C → A. C is the key of R3 → in BCNF.

R4(B, C) – no non-trivial FD holds on {B,C} alone → in BCNF (trivially, since B and C together are the only key).

Final BCNF Decomposition

Relation	Attributes	Key
R1	C, D	C

R3	A, C	C
R4	B, C	B, C

Justification

The decomposition {R1, R3, R4} is lossless-join: C is common to all three and functionally determines D and A, so the original relation can be reconstructed by natural joins on C.

It is not fully dependency-preserving – the FD $AB \rightarrow C$ cannot be checked without joining R3 and R4 back together. This is an expected, well-known trade-off of BCNF: BCNF guarantees no redundancy from FDs but does not guarantee every original FD is enforceable locally in a single relation.

Every relation in the final set now satisfies BCNF, since in each remaining FD the left-hand side is a key of its relation.

Q4. Views and Constraints for Banking System Security

Scenario

A banking system requires secure, role-based data access. Views and constraints are designed to protect data integrity and restrict what different categories of users (customers, tellers, managers) can see and do.

Base Schema

Customer (CustomerID PK, Name, Address, SSN, DOB)
 Account (AccountNo PK, CustomerID FK, AccountType, Balance, BranchID)
 Transaction(TxnID PK, AccountNo FK, TxnType, Amount, TxnDate)
 Employee (EmpID PK, Name, Role, BranchID)

Integrity Constraints

```
ALTER TABLE Account
ADD CONSTRAINT chk_balance CHECK (Balance >= 0);

ALTER TABLE Account
ADD CONSTRAINT fk_cust FOREIGN KEY (CustomerID)
REFERENCES Customer(CustomerID) ON DELETE RESTRICT;

ALTER TABLE Transaction
ADD CONSTRAINT chk_amount CHECK (Amount > 0),
ADD CONSTRAINT fk_acct FOREIGN KEY (AccountNo)
REFERENCES Account(AccountNo) ON DELETE CASCADE;
```

```
ALTER TABLE Customer
ADD CONSTRAINT uq_ssn UNIQUE (SSN),
MODIFY Name VARCHAR(100) NOT NULL;
```

Views for Different User Roles

1. Customer self-service view – exposes only the logged-in customer's own account summary, hiding SSN and other customers' data entirely:

```
CREATE VIEW CustomerAccountView AS
SELECT a.AccountNo, a.AccountType, a.Balance
FROM Account a
WHERE a.CustomerID = CURRENT_USER_ID(); -- resolved by the application layer
```

2. Teller view – can see account balances for day-to-day transactions but not full customer SSN/DOB:

```
CREATE VIEW TellerView AS
SELECT c.CustomerID, c.Name, a.AccountNo, a.AccountType, a.Balance, a.BranchID
FROM Customer c JOIN Account a ON c.CustomerID = a.CustomerID;
```

3. Manager view – full analytical access including sensitive fields, restricted to their own branch:

```
CREATE VIEW ManagerView AS
SELECT c.*, a.*
FROM Customer c JOIN Account a ON c.CustomerID = a.CustomerID
WHERE a.BranchID = CURRENT_BRANCH_ID();
```

Access Control (GRANT)

```
GRANT SELECT ON CustomerAccountView TO ROLE customer;
GRANT SELECT, INSERT ON TellerView TO ROLE teller;
GRANT SELECT ON ManagerView TO ROLE manager;
REVOKE ALL ON Customer, Account, Transaction FROM ROLE customer, teller;
```

Justification

Views enforce column-level and row-level security: customers never see the base tables directly, only a filtered view scoped to their own row, which prevents horizontal privilege

escalation.

CHECK constraints (Balance >= 0, Amount > 0) enforce business rules at the data layer so no application bug can insert an invalid state.

Foreign keys with ON DELETE RESTRICT/CASCADE preserve referential integrity between customers, accounts and transactions.

GRANT/REVOKE on the views (not the base tables) ensures the principle of least privilege – each role only receives the minimum access needed for its function.

Q5. Normalize a Redundant Sales Database (3NF/BCNF)

Scenario

A sales database contains redundant data leading to anomalies. The schema is analyzed and normalized up to 3NF/BCNF to eliminate redundancy.

Assumed Unnormalized Table

Sales (OrderID, OrderDate, CustomerID, CustomerName, CustomerCity, ProductID, ProductName, UnitPrice, Quantity, SalesRepID, SalesRepName)

Primary key: (OrderID, ProductID) – one row per product line within an order.

Anomalies in the Unnormalized Table

Anomaly	Example
Insertion	A new product cannot be added to the catalog until it appears on an actual order line.
Update	If a customer's city changes, every order row for that customer must be updated, risking inconsistency.
Deletion	Deleting the only order placed by a customer erases that customer's contact details entirely.

Functional Dependencies

CustomerID -> CustomerName, CustomerCity
ProductID -> ProductName, UnitPrice
SalesRepID -> SalesRepName
OrderID -> OrderDate, CustomerID, SalesRepID
(OrderID, ProductID) -> Quantity

Normalized (3NF/BCNF) Decomposition

Relation	Attributes	Key
Customer	CustomerID, CustomerName, CustomerCity	PK: CustomerID
Product	ProductID, ProductName, UnitPrice	PK: ProductID
SalesRep	SalesRepID, SalesRepName	PK: SalesRepID
Orders	OrderID, OrderDate, CustomerID (FK), SalesRepID (FK)	PK: OrderID
OrderDetails	OrderID (FK), ProductID (FK), Quantity	PK: (OrderID, ProductID)

Justification

Every non-key attribute now depends only on the primary key of its own relation, with no partial or transitive dependencies – each relation is in BCNF since every determinant (CustomerID, ProductID, SalesRepID, OrderID, and the composite key) is a candidate key of its relation.

Customer, product and sales-rep details are stored exactly once, removing update anomalies.

New products/customers/ reps can be inserted independently of order activity, removing insertion anomalies.

Deleting an order no longer risks losing master data about a customer or product, removing deletion anomalies.

Q6. Joins and Subqueries for Frequent Customers

Scenario

Given a dataset of customers and orders, SQL queries using joins and subqueries retrieve meaningful information such as frequent customers.

Assumed Schema

Customer (CustomerID PK, Name, City)

Orders (OrderID PK, CustomerID FK, OrderDate, Amount)

Query 1 – Customers with more than 5 orders (JOIN + GROUP BY)

```
SELECT c.CustomerID, c.Name, COUNT(o.OrderID) AS TotalOrders
FROM Customer c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING COUNT(o.OrderID) > 5
ORDER BY TotalOrders DESC;
```

Query 2 – Customers whose total spend exceeds the average (Subquery)

```
SELECT c.CustomerID, c.Name, SUM(o.Amount) AS TotalSpend
FROM Customer c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING SUM(o.Amount) > (
    SELECT AVG(CustomerTotal) FROM (
        SELECT SUM(Amount) AS CustomerTotal
        FROM Orders
        GROUP BY CustomerID
    ) AS PerCustomer
);
```

Query 3 – Top 5 most frequent customers (correlated subquery, no LIMIT dependency)

```
SELECT c.CustomerID, c.Name,
    (SELECT COUNT(*) FROM Orders o WHERE o.CustomerID = c.CustomerID) AS
OrderCount
FROM Customer c
WHERE (SELECT COUNT(*) FROM Orders o WHERE o.CustomerID = c.CustomerID) >= 3
ORDER BY OrderCount DESC;
```

Justification

JOIN + GROUP BY + HAVING is the standard set-based approach for counting/summing related rows per customer and filtering on the aggregate – more efficient than row-by-row procedural logic.

The nested subquery in Query 2 first computes each customer's total, then averages those totals, which is different (and correct) compared to simply averaging individual

order Amount rows.

A correlated subquery (Query 3) is shown as an alternative to JOIN + GROUP BY for cases where per-row scalar counts are needed alongside other non-aggregated columns.

Q7. Multi-Valued Dependencies – Decompose to 4NF

Scenario

A relation contains multi-valued dependencies. The structure is analyzed and decomposed into 4NF with reasoning.

Given Relation

EmployeeSkillLanguage (EmpID, Skill, Language)

An employee can have several skills and, independently, speak several languages – the set of skills and the set of languages an employee has are unrelated to each other. Example instance:

EmpID	Skill	Language
E1	Java	English
E1	Java	Tamil
E1	Python	English
E1	Python	Tamil

Every combination of E1's skills with E1's languages must appear for the table to be consistent – this repetition is the signature of a multi-valued dependency.

Multi-Valued Dependencies

EmpID ->-> Skill
EmpID ->-> Language

Why this violates 4NF

The only candidate key of EmployeeSkillLanguage is the full set (EmpID, Skill, Language).

The MVD EmpID ->-> Skill is non-trivial (Language is not a subset of {EmpID, Skill}) and

EmpID is not a superkey, so the relation violates 4NF even though it may already be in BCNF (there are no non-trivial functional dependencies to violate).

4NF Decomposition

EmployeeSkill (EmpID, Skill) PK: (EmpID, Skill)
EmployeeLanguage (EmpID, Language) PK: (EmpID, Language)

Justification

Splitting on the independent multi-valued facts removes the need to store every skill x

language combination – EmployeeSkill and EmployeeLanguage each store one independent fact per row.

The decomposition is lossless-join: rejoining EmployeeSkill and EmployeeLanguage on EmpID reconstructs the original Cartesian-product rows exactly, because that Cartesian product is precisely what the MVD asserted must hold.

Both resulting relations trivially satisfy 4NF, since in each, the key (EmpID, Skill) or (EmpID, Language) is the only non-trivial determinant.

Q8. Evaluating Improper Constraints and Suggesting Fixes

Scenario

A database designer used improper constraints, leading to inconsistent data. The schema is evaluated and appropriate integrity constraints are suggested.

Flawed Schema (as designed)

```
CREATE TABLE Employee (  
  EmpID INT,  
  Name VARCHAR(50),  
  Email VARCHAR(50),  
  DeptID INT,  
  Salary INT,  
  ManagerID INT  
);
```

Evaluation – Issues Identified

Issue	Consequence
No PRIMARY KEY on EmpID	Duplicate employee records can be inserted; rows cannot be uniquely referenced by other tables.
No NOT NULL on Name/Email	Employees can be saved with missing names or emails.
No UNIQUE constraint on Email	Two employees can share the same email, breaking login/notification logic.
No FOREIGN KEY from DeptID to Department	Salary/DeptID can reference a department that does not exist (orphan reference).
No CHECK on Salary	Negative or zero salaries can be inserted.
No self-referential FK on ManagerID	ManagerID can point to a non-existent EmpID.

Corrected Schema

```
CREATE TABLE Employee (
  EmpID   INT PRIMARY KEY,
  Name    VARCHAR(50) NOT NULL,
  Email   VARCHAR(50) NOT NULL UNIQUE,
  DeptID  INT NOT NULL,
  Salary  INT CHECK (Salary > 0),
  ManagerID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
  FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID)
);
```

Justification

PRIMARY KEY guarantees entity integrity – every employee row is uniquely and reliably identifiable.

NOT NULL and UNIQUE on Email enforce that every employee has exactly one valid, distinct contact address.

FOREIGN KEY constraints enforce referential integrity so DeptID and ManagerID can never point to non-existent rows.

CHECK (Salary > 0) is a domain constraint that stops invalid business data from ever entering the table, rather than relying on application-layer validation alone.

Q9. Join Dependencies – Decompose into 5NF

Scenario

For a given relational schema, the presence of join dependencies is analyzed and the relation is decomposed into 5NF to remove redundancy.

Given Relation

SupplierPartProject (Supplier, Part, Project)

This is the classical ternary relationship: a Supplier supplies a Part to a Project, but the fact cannot be derived from any two of the three attributes alone – it is only true as a genuine three-way combination. Example instance illustrating the redundancy if kept as one table:

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S2	P1	J1
S1	P1	J2

Join Dependency

JD: $\star(\{Supplier, Part\}, \{Part, Project\}, \{Supplier, Project\})$

This states that SupplierPartProject can be losslessly reconstructed by joining its three binary projections back together, but not by any single pairwise projection. If this join dependency holds and does not follow from the relation's candidate key (the full attribute set), the relation is not in 5NF.

5NF Decomposition

SP (Supplier, Part) PK: (Supplier, Part)
PJ (Part, Project) PK: (Part, Project)
SJ (Supplier, Project) PK: (Supplier, Project)

Justification

Storing the three binary relationships separately removes the redundancy that arises from repeating full Supplier-Part-Project triples whenever any one dimension has multiple valid combinations.

The three-way natural join SP PJ SJ reconstructs the original relation exactly, satisfying losslessness.

Each of SP, PJ, SJ is all-key (the two attributes together form the only key) with no further non-trivial join dependency, so all three are in 5NF.

Caution: this decomposition is only correct if the ternary fact truly decomposes into three independent pairwise facts. If the original triples cannot be reconstructed from the three pairwise joins without introducing spurious combinations, the relation was already in 5NF and should not be decomposed this way – the join dependency must be verified against the actual data before decomposing.

Q10. Analyzing and Optimizing an Inefficient SQL Query

Scenario

A system uses inefficient SQL queries for data retrieval. The queries are analyzed and optimized using joins, indexing concepts, or query restructuring.

Inefficient Query (as given)

```
SELECT *  
FROM Orders  
WHERE CustomerID IN (  
    SELECT CustomerID FROM Customer WHERE City = 'Chennai'
```

```
)  
AND OrderDate > '2025-01-01';
```

Issues Identified

Issue	Why it is inefficient
SELECT *	Fetches every column, including large/unused ones, increasing I/O and network transfer.
Uncorrelated IN subquery	Many optimizers re-evaluate or materialize the subquery per outer row without a good index, which does not scale on large Customer tables.
No index on Orders.CustomerID / OrderDate	Forces a full table scan of Orders instead of an index seek/range scan.
No index on Customer.City	Forces a full table scan to find Chennai customers before the semi-join can even begin.

Optimized Query (restructured as a JOIN)

```
SELECT o.OrderID, o.OrderDate, o.Amount, c.Name  
FROM Orders o  
JOIN Customer c  
  ON o.CustomerID = c.CustomerID  
WHERE c.City = 'Chennai'  
AND o.OrderDate > '2025-01-01';
```

Supporting Indexes

```
CREATE INDEX idx_customer_city ON Customer(City);  
CREATE INDEX idx_orders_custdate ON Orders(CustomerID, OrderDate);
```

Justification:

Replacing IN (subquery) with an explicit JOIN lets the query optimizer choose an efficient join strategy (hash join / index nested-loop) rather than treating the subquery as an opaque black box.

Selecting only the needed columns instead of SELECT * reduces I/O and avoids unnecessary data transfer, which matters more as row counts grow.

idx_customer_city lets the engine locate Chennai customers via an index seek instead of a full scan.

The composite index idx_orders_custdate on (CustomerID, OrderDate) supports

both the join predicate and the date range filter in a single index scan, since CustomerID is the leading (equality-friendly) column and OrderDate the trailing range column. Together, these changes move the query from full table scans on both tables to index-driven access, which is the dominant cost saving for large datasets.